

Servidor de sockets multicliente con envío de correo electrónico mediante un sistema de mensajería externo

Práctica de Sistemas Distribuidos (Tema 2: Comunicación)

Guillermo Romo Sánchez

Grado en Robótica e Inteligencia Artificial

Escuela Politécnica, Universidad Camilo José Cela

28 de septiembre de 2026

Índice

1. Introducción	3
2. Objetivos	3
3. Fundamentos teóricos	4
3.1. Sockets TCP y modelo cliente-servidor	4
3.2. TCP como flujo de bytes: delimitación de mensajes	4
3.3. Concurrencia en el servidor	5
3.4. Delegación en un sistema de mensajería externo (MOM)	5
3.5. Direcciones de escucha: 0.0.0.0 frente a 127.0.0.1	6
3.6. Proxy TCP en plataformas PaaS	6
4. Diseño	6
4.1. Protocolo de aplicación	6
4.2. Configuración por variables de entorno	6
4.3. Modo de prueba	7
5. Implementación	7
5.1. Estructura del proyecto	7
5.2. Servidor (<code>server.py</code>)	7
5.3. Cliente (<code>client.py</code>)	11
5.4. Prueba automática multicliente (<code>prueba_multicliente.py</code>)	12
6. Desarrollo de la práctica: despliegue	13
6.1. Elección de plataforma	13
6.2. Método de despliegue	13
6.3. Configuración del proxy TCP	14
7. Resultados	14

8. Análisis	15
8.1. Sobre la naturaleza de los errores de red observados	15
8.2. Sobre el filtrado de puertos SMTP en plataformas de hosting	16
8.3. Sobre delegar en un servicio externo vía HTTP en lugar de SMTP directo .	16
8.4. Sobre la concurrencia del servidor	17
9. Problemas encontrados	17
10.Soluciones adoptadas	18
11.Conclusiones	19
12.Bibliografía	19
13.Anexos	20
13.1. Dockerfile	20
13.2. Procfile	20
13.3. requirements.txt	20
13.4. runtime.txt	20
13.5. Registro real del despliegue exitoso (railway up)	20
13.6. Sesión real del cliente confirmando el envío de correo	21

1. Introducción

Esta memoria documenta el diseño, la implementación y el despliegue de un servidor TCP multiciente desarrollado para la asignatura de Sistemas Distribuidos, dentro del bloque dedicado a la comunicación entre procesos (Tema 2). El servidor parte del ejemplo básico visto en clase, construido sobre la interfaz de sockets de Berkeley (`socket`, `bind`, `listen`, `accept`, `send/recv`), y lo amplía en tres aspectos que la propia diapositiva del ejemplo básico señala como limitaciones: la atención a un único cliente, el tratamiento de los datos como bytes sueltos en lugar de mensajes completos, y la ausencia de cualquier interacción con un sistema externo.

Sobre esa base se ha implementado un pequeño protocolo de aplicación orientado a texto, con varios comandos, entre ellos uno que envía un correo electrónico real delegando en un sistema de mensajería externo. Esto último sirve para ilustrar un patrón habitual en sistemas distribuidos: un servicio construido sobre un protocolo (aquí, sockets TCP con un protocolo propio) que, para completar una petición del cliente, se apoya en otro sistema de mensajería (un MOM, en la terminología del tema) en lugar de implementar esa funcionalidad por completo desde cero.

Además del código, esta memoria recoge el proceso real de despliegue del servidor en una plataforma de hosting en la nube (Railway), que resultó bastante más instructivo de lo previsto: la plataforma introdujo varias restricciones de red no documentadas explícitamente que obligaron a diagnosticar y corregir el sistema en varias iteraciones. Esa secuencia completa, con los resultados reales observados en cada paso, se documenta en las secciones de resultados, problemas encontrados y soluciones adoptadas, distinguiendo siempre entre lo que se esperaba teóricamente y lo que se obtuvo en la ejecución real.

2. Objetivos

- Implementar un servidor TCP capaz de atender a varios clientes simultáneamente, sin bloquearse en la atención de uno solo.
- Diseñar un protocolo de aplicación sencillo sobre TCP, resolviendo el problema de la delimitación de mensajes en un protocolo de transporte que solo entrega un flujo de bytes.
- Integrar el servidor con un sistema de mensajería externo (correo electrónico) para completar una petición del cliente, como ejemplo de un servicio construido sobre sockets que se apoya en un MOM externo.
- Desplegar el servidor en una plataforma de hosting en la nube, de forma que sea accesible públicamente desde cualquier cliente TCP.
- Diagnosticar y documentar, con evidencia real de ejecución, los problemas de red y de plataforma surgidos durante el despliegue, distinguiendo sus causas a distintos niveles (enrutamiento, filtrado de puertos, protección a nivel de aplicación).

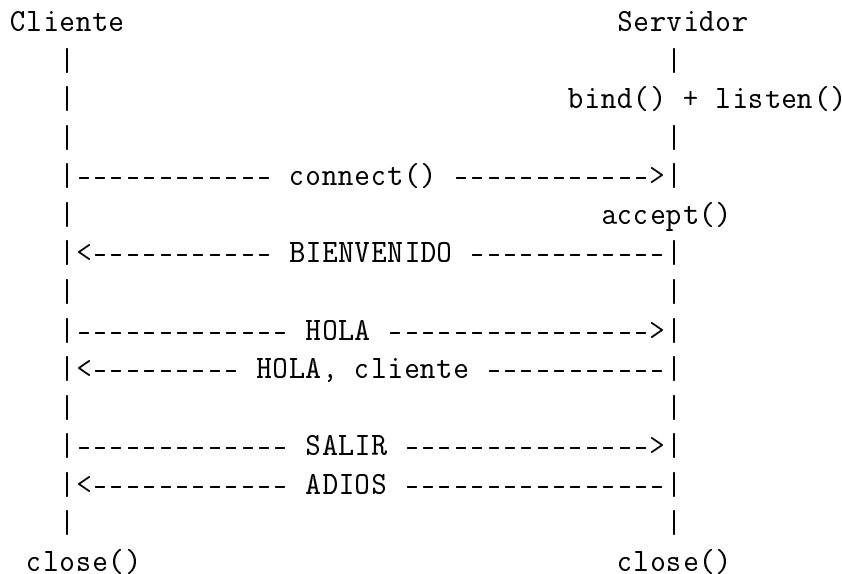
3. Fundamentos teóricos

3.1. Sockets TCP y modelo cliente-servidor

La comunicación se realiza mediante sockets TCP (`SOCK_STREAM`), que proporcionan un servicio de transporte orientado a conexión, fiable y con entrega ordenada de los datos. El servidor sigue la secuencia clásica de un servidor TCP:

1. **socket()**: crea el descriptor del socket.
2. **bind()**: lo asocia a una dirección local y un puerto.
3. **listen()**: lo pone en modo escucha, con una cola de conexiones pendientes.
4. **accept()**: bloquea hasta que llega una conexión entrante, y devuelve un nuevo socket dedicado a esa conexión.
5. **recv()** / **send()**: intercambio de datos sobre la conexión ya establecida.
6. **close()**: cierre de la conexión.

El cliente, por su parte, solo necesita **connect()** para establecer la conexión contra la dirección y puerto del servidor, y a partir de ahí puede intercambiar datos con **send()/recv()**.



3.2. TCP como flujo de bytes: delimitación de mensajes

TCP no conoce el concepto de "mensaje": entrega un flujo continuo de bytes, sin marcar dónde empieza o termina cada unidad lógica de la aplicación. Esto tiene dos consecuencias prácticas que hay que resolver en la capa de aplicación:

- Una llamada a **recv()** puede devolver menos bytes de los que el emisor envió en una sola llamada a **send()** (el mensaje llega "partido").
- Una llamada a **recv()** puede devolver los bytes de varios mensajes enviados por separado, si llegaron juntos al búfer del receptor (el mensaje llega "pegado.^a otro).

La solución adoptada en esta práctica es un delimitador explícito: cada mensaje del protocolo termina en un salto de línea ("`\n`"). El receptor acumula los bytes recibidos en un búfer y solo procesa un mensaje cuando encuentra ese delimitador, dejando en el búfer cualquier resto que pertenezca al siguiente mensaje. Esta es la técnica más simple de "framing."^{en} protocolos de texto, y es la misma que usan por ejemplo SMTP o HTTP/1.1 en sus líneas de cabecera.

3.3. Concurrencia en el servidor

Un servidor que solo llama a `accept()` una vez queda bloqueado atendiendo a ese único cliente hasta que termine la conversación, y no puede aceptar nuevas conexiones mientras tanto. La solución aplicada es lanzar un hilo (`threading.Thread`) por cada conexión aceptada, de forma que el hilo principal vuelve inmediatamente a `accept()` para poder admitir el siguiente cliente, mientras cada hilo secundario atiende su propia conversación de forma independiente. Los hilos se marcan como `daemon=True` para que no impidan la finalización del proceso si el servidor se detiene.

Es importante no confundir esta concurrencia (varios hilos del mismo proceso, en la misma máquina, compartiendo memoria) con el propio carácter distribuido del sistema (cliente y servidor son procesos independientes, potencialmente en máquinas distintas, que solo se comunican por mensajes a través de la red, sin memoria compartida). Son dos fenómenos distintos que coexisten en esta práctica: concurrencia a nivel de proceso servidor, y comunicación distribuida a nivel de la relación cliente-servidor.

3.4. Delegación en un sistema de mensajería externo (MOM)

El comando `CORREO` no implementa el envío de correo dentro del propio servidor de sockets: delega esa tarea en un sistema externo especializado en mensajería. Este patrón, en el que un servicio se apoya en un middleware orientado a mensajes (MOM) para completar parte de su trabajo, es habitual en sistemas distribuidos reales, donde raramente conviene reimplementar desde cero un protocolo de entrega de mensajes ya resuelto por otro sistema.

En esta práctica se han usado, en momentos distintos, dos protocolos de transporte diferentes hacia ese sistema externo:

- **SMTP** (Simple Mail Transfer Protocol, RFC 5321): protocolo textual sobre TCP, pensado específicamente para la transmisión de correo entre servidores. Puede cifrarse de dos formas: **STARTTLS** (RFC 3207), en la que la conexión empieza en texto claro por el puerto 587 y se eleva a TLS mediante un comando explícito antes de autenticar, o **TLS implícito**, en la que la conexión se cifra desde el primer byte, tradicionalmente por el puerto 465.
- **Una API HTTP** (en este caso, la de Resend): el servidor hace una petición HTTPS normal (método POST, cuerpo en JSON) a un servicio externo, que es quien finalmente entrega el correo por SMTP en su propia infraestructura. El servidor de la práctica ya no habla SMTP en absoluto: solo HTTP sobre TLS.

Como se explica en la sección de problemas encontrados, el cambio del primer mecanismo al segundo no fue una decisión de diseño inicial, sino una solución adoptada tras comprobar empíricamente que el primero no era viable en el entorno de despliegue elegido.

3.5. Direcciones de escucha: 0.0.0.0 frente a 127.0.0.1

El servidor se vincula a 0.0.0.0, una dirección especial que no identifica una interfaz de red concreta sino que indica "escuchar en todas las interfaces disponibles de esta máquina". Es distinta de 127.0.0.1 (*localhost*), que solo acepta conexiones originadas en la propia máquina. En un contenedor desplegado en la nube, el servidor debe escuchar en 0.0.0.0 para que el proxy de la plataforma (que técnicamente se conecta desde otra interfaz interna del propio contenedor) pueda alcanzarlo; si escuchara solo en 127.0.0.1, sería inaccesible desde fuera del propio proceso.

3.6. Proxy TCP en plataformas PaaS

Railway, como otras plataformas de *Platform as a Service*, no expone directamente el puerto interno del contenedor a Internet. En su lugar, ofrece un **proxy TCP**: un servicio de la propia plataforma que escucha en un host y puerto públicos, y reenvía el tráfico al puerto interno indicado del contenedor (en este caso, el 5050 en el que escucha `server.py`). Sin ese proxy configurado explícitamente, no existe ninguna ruta pública hacia el servicio, aunque el proceso esté en ejecución con normalidad dentro del contenedor.

4. Diseño

4.1. Protocolo de aplicación

El protocolo diseñado es textual, orientado a líneas, y sigue un esquema petición-respuesta síncrono: el cliente envía una línea, el servidor responde con otra (o varias) antes de que el cliente envíe la siguiente. No hay mensajes asíncronos ni notificaciones espontáneas del servidor.

Comando	Comportamiento
HOLA	Responde HOLA, cliente
HORA	Responde con la fecha y hora actuales del servidor
ECO <mensaje>	Devuelve el mismo mensaje recibido
CORREO <destino> <asunto> <cuerpo>	Envía un correo real a través del sistema de mensajería ext
SALIR	Responde ADIOS y cierra la conexión
(cualquier otro)	Responde ERROR comando no reconocido

Cuadro 1: Comandos del protocolo de aplicación

El comando `CORREO` usa `|` como separador de sus tres argumentos (destinatario, asunto y cuerpo), ya que el espacio no sirve como separador porque el asunto y el cuerpo pueden contener espacios.

4.2. Configuración por variables de entorno

Todos los parámetros que varían entre el entorno local y el entorno de despliegue (dirección de escucha, puerto, credenciales del servicio de correo) se leen de variables de entorno, con valores por defecto razonables para ejecución local. Esto permite desplegar exactamente el mismo código, sin modificarlo, en distintas plataformas: solo cambia la

configuración externa. Las credenciales en particular nunca se escriben en el código fuente ni se suben al repositorio: se gestionan como variables/secretos desde el panel de la plataforma de hosting.

4.3. Modo de prueba

Si las credenciales del sistema de mensajería no están definidas (por ejemplo, en un entorno de desarrollo local sin acceso a ellas), el servidor entra en `MODO_PRUEBA`: el comando `CORREO` sigue respondiendo con el protocolo completo, pero simula el envío en lugar de contactar con el servicio externo. Esto permite probar la lógica del protocolo sin necesidad de credenciales reales ni de conexión a Internet.

5. Implementación

5.1. Estructura del proyecto

<code>server.py</code>	Servidor TCP multicliente (codigo principal)
<code>client.py</code>	Cliente interactivo de linea de comandos
<code>prueba_multicliente.py</code>	Prueba automatica con dos clientes en paralelo
<code>Dockerfile</code>	Imagen del contenedor para el despliegue
<code>Procfile</code>	Comando de arranque (alternativa Nixpacks)
<code>requirements.txt</code>	Sin dependencias externas (solo biblioteca estandar)
<code>runtime.txt</code>	Version de Python (3.11)

5.2. Servidor (`server.py`)

Listing 1: Servidor TCP multicliente con envío de correo mediante API HTTP externa

```
1  """
2  Servidor TCP multicliente con un pequeno protocolo de comandos.
3  Practica de Sistemas Distribuidos: comunicacion mediante sockets (T2).
4
5  Este servidor parte del ejemplo visto en clase (socket + bind + listen +
6  accept + recu/send) y lo amplia en tres puntos, precisamente los tres
7  limites que la diapositiva senala en el servidor basico:
8
9  1. Atiende a varios clientes a la vez: un hilo por conexion, en lugar
10     de bloquearse en un unico accept() tras el primer cliente.
11  2. Trabaja con mensajes completos y no con bytes sueltos: como TCP
12     entrega un flujo de bytes sin fronteras propias, el servidor arma
13     cada mensaje acumulando en un buffer hasta encontrar un salto de
14     linea, que actua como delimitador del protocolo de aplicacion.
15  3. Incorpora un comando que envia un correo electronico real, para
16     mostrar como un servicio construido sobre sockets puede apoyarse en
17     un sistema de mensajeria externo (un MOM, en la terminologia del
18     tema 2) para completar una peticion del cliente.
19
20  Sobre el mecanismo de envio de correo: la primera version de esta
21  practica usaba SMTP directo (conexion de socket al puerto 587/465 de un
22  servidor de correo). Al desplegar en Railway se comprobo empiricamente
23  que la plataforma bloquea el trafico saliente hacia esos puertos (la
24  conexion agota el tiempo de espera sin respuesta), una restriccion
25  habitual en plataformas de hosting gratuitas para evitar que sus
```

```

26 contenedores se usen para enviar spam. La solucion adoptada es enviar el
27 correo a traves de la API HTTP de Resend (https://resend.com): en lugar
28 de que el servidor hable el protocolo SMTP directamente, hace una
29 peticion HTTPS (puerto 443, sin restriccion en Railway) a un servicio
30 externo que es quien finalmente entrega el correo por SMTP. Sigue siendo
31 el mismo concepto (delegar en un sistema de mensajeria externo), con un
32 protocolo de transporte distinto hacia ese sistema.
33
34 HOST y PORT son configurables por variable de entorno para poder
35 desplegar el mismo codigo sin tocarlo en distintas plataformas (Railway,
36 Fly.io, tu propio equipo...), que asignan el puerto interno de formas
37 distintas.
38 """
39
40 from socket import socket, AF_INET, SOCK_STREAM, SOL_SOCKET, SO_REUSEADDR
41 import threading
42 import datetime
43 import json
44 import os
45 import urllib.request
46 import urllib.error
47
48 HOST = os.environ.get("SERVER_HOST", "0.0.0.0") # escucha en todas las interfaces
49 PORT = int(os.environ.get("SERVER_PORT", os.environ.get("PORT", "5050")))
50
51 # --- Configuracion del servicio de correo (API HTTP de Resend) -----
52 # La clave de API nunca se escribe en el codigo fuente: se lee de una
53 # variable de entorno (en el hosting, se configura como "secret" o
54 # "variable" desde el panel, nunca en el repositorio). Si no esta
55 # definida, el servidor entra en MODO_PRUEBA y simula el envio en lugar
56 # de hacer una peticion real.
57 RESEND_API_KEY = os.environ.get("RESEND_API_KEY")
58 # Direccion remitente. "onboarding@resend.dev" es la direccion de prueba
59 # que ofrece Resend sin necesidad de verificar un dominio propio; en modo
60 # sandbox (sin dominio verificado) solo se puede enviar correo a la
61 # direccion con la que se creo la cuenta de Resend.
62 RESEND_FROM = os.environ.get("RESEND_FROM", "onboarding@resend.dev")
63 MODO_PRUEBA = RESEND_API_KEY is None
64
65
66 def enviar_correo(destinatario, asunto, cuerpo):
67     """
68     Envia un correo mediante una peticion HTTPS POST a la API de Resend
69     (https://api.resend.com/emails), autenticada con la cabecera
70     "Authorization: Bearer <clave>". El cuerpo de la peticion va en JSON,
71     tal como exige esa API.
72
73     En MODO_PRUEBA no se abre ninguna conexcion de red: se registra por
74     consola lo que se habria enviado, para poder probar el protocolo
75     completo (el comando CORREO y su respuesta) sin credenciales reales.
76     """
77     if MODO_PRUEBA:
78         print(f"[MODO_PRUEBA] Simulando envio -> Para: {destinatario} "
79               f"| Asunto: {asunto} | Cuerpo: {cuerpo}")
80         return True, "simulado (MODO_PRUEBA activo, sin RESEND_API_KEY)"
81
82     cuerpo_peticion = json.dumps({
83         "from": RESEND_FROM,

```

```

84     "to": [destinatario],
85     "subject": asunto,
86     "text": cuerpo,
87 }).encode("utf-8")
88
89 peticion = urllib.request.Request(
90     "https://api.resend.com/emails",
91     data=cuerpo_peticion,
92     method="POST",
93     headers={
94         "Authorization": f"Bearer {RESEND_API_KEY}",
95         "Content-Type": "application/json",
96         # Cloudflare (que protege la API de Resend) bloquea con un 403
97         # (error code: 1010) las peticiones que usan el User-Agent por
98         # defecto de urllib ("Python-urllib/x.y"), por ser una firma
99         # facilmente identificable como trafico de script. Un
100         # User-Agent propio evita ese bloqueo.
101         "User-Agent": "practica-sistemas-distribuidos-sockets/1.0",
102     },
103 )
104
105 try:
106     with urllib.request.urlopen(peticion, timeout=10) as respuesta:
107         datos_respuesta = json.loads(respuesta.read().decode("utf-8"))
108         return True, f"correo enviado (id Resend: {datos_respuesta.get('id', '?')})"
109
110 except urllib.error.HTTPError as error:
111     detalle = error.read().decode("utf-8", errors="replace")
112     return False, f"error HTTP {error.code} de Resend: {detalle}"
113 except urllib.error.URLError as error:
114     return False, f"error de red: {error.reason}"
115
116 class Servidor:
117     def __init__(self, host=HOST, port=PORT):
118         self.host = host
119         self.port = port
120         self.sock = socket(AF_INET, SOCK_STREAM)
121         self.sock.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
122
123     def iniciar(self):
124         self.sock.bind((self.host, self.port))
125         self.sock.listen(5)
126         print(f"Servidor escuchando en {self.host}:{self.port} "
127             f"(MODO_PRUEBA={'activo' if MODO_PRUEBA else 'inactivo'})")
128         try:
129             while True:
130                 conn, addr = self.sock.accept()
131                 hilo = threading.Thread(
132                     target=self.atender_cliente, args=(conn, addr), daemon=True
133                 )
134                 hilo.start()
135         except KeyboardInterrupt:
136             print("\nCerrando servidor...")
137         finally:
138             self.sock.close()
139
140     def atender_cliente(self, conn, addr):

```

```

141 print(f"[+] Conexion aceptada de {addr}")
142 buffer = ""
143 try:
144     with conn:
145         conn.sendall(
146             b"BIENVENIDO. Comandos: HOLA | HORA | ECO <msg> | "
147             b"CORREO <destino>|<asunto>|<cuerpo> | SALIR\n"
148         )
149     while True:
150         datos = conn.recv(1024)
151         if not datos:
152             # recv vacio: el cliente cerro la conexion
153             break
154         buffer += datos.decode("utf-8", errors="replace")
155         # Un mensaje termina en salto de linea: puede llegar
156         # partido en varios recv, o varios mensajes en un recv.
157         while "\n" in buffer:
158             linea, buffer = buffer.split("\n", 1)
159             if not self.procesar_linea(conn, linea.strip(), addr):
160                 return
161     except ConnectionResetError:
162         print(f"[!] {addr} cerro la conexion de forma abrupta")
163     finally:
164         print(f"[-] Conexion cerrada con {addr}")
165
166 def procesar_linea(self, conn, linea, addr):
167     """Interpreta un comando y responde. Devuelve False si hay que cerrar."""
168     if not linea:
169         return True
170
171     partes = linea.split(" ", 1)
172     comando = partes[0].upper()
173     resto = partes[1] if len(partes) > 1 else ""
174
175     if comando == "HOLA":
176         conn.sendall(b"HOLA, cliente\n")
177
178     elif comando == "HORA":
179         ahora = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
180         conn.sendall(f"{ahora}\n".encode("utf-8"))
181
182     elif comando == "ECO":
183         conn.sendall(f"{resto}\n".encode("utf-8"))
184
185     elif comando == "CORREO":
186         try:
187             destinatario, asunto, cuerpo = resto.split("|", 2)
188         except ValueError:
189             conn.sendall(b"ERROR formato: CORREO <destino>|<asunto>|<cuerpo>\n")
190             return True
191         ok, detalle = enviar_correo(destinatario.strip(), asunto.strip(), cuerpo.strip())
192         estado = "OK" if ok else "ERROR"
193         conn.sendall(f"{estado} {detalle}\n".encode("utf-8"))
194
195     elif comando == "SALIR":
196         conn.sendall(b"ADIOS\n")
197     return False

```

```

198
199         else:
200             conn.sendall(b"ERROR comando no reconocido\n")
201
202         return True
203
204
205 if __name__ == "__main__":
206     Servidor().iniciar()

```

Los puntos más relevantes de la implementación:

- **Framing de mensajes** (líneas 149-160): el método `atender_cliente` acumula los bytes recibidos en la variable `buffer` y extrae líneas completas con `buffer.split("\n", 1)` en un bucle, de forma que un mensaje partido entre dos llamadas a `recv()` se reconstruye correctamente, y varios mensajes llegados en una sola llamada se procesan uno a uno.
- **Concurrencia**: cada conexión aceptada se atiende en un hilo (`daemon=True`) independiente, de forma que el bucle principal de `iniciar()` puede volver inmediatamente a `accept()`.
- **Envío de correo mediante API HTTP**: la función `enviar_correo` construye una petición HTTPS con `urllib.request` hacia la API de Resend, con el cuerpo en JSON y autenticación mediante cabecera `Authorization: Bearer`. Se explica en la sección de problemas encontrados por qué se llegó a esta solución en lugar de usar SMTP directo, que fue el diseño inicial.
- **Manejo de errores**: se distinguen los errores HTTP devueltos por la API externa (`urllib.error.HTTPError`) de los errores de red de bajo nivel (`urllib.error.URLError`), devolviendo en cada caso un mensaje distinto al cliente.

5.3. Cliente (`client.py`)

Listing 2: Cliente interactivo de línea de comandos

```

1  """
2  Cliente interactivo para el servidor de sockets con envio de correo.
3  Practica de Sistemas Distribuidos: comunicacion mediante sockets (T2).
4
5  Uso:
6      python client.py -> conecta a 127.0.0.1:5050
7      python client.py mi-servidor.up.railway.app 12345 -> conecta al host y
8                                          puerto publicos del despliegue
9  """
10
11 import sys
12 from socket import socket, AF_INET, SOCK_STREAM
13
14 HOST = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
15 PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 5050
16
17
18 def main():
19     with socket(AF_INET, SOCK_STREAM) as s:

```

```

20     print(f"Conectando a {HOST}:{PORT}...")
21     s.connect((HOST, PORT))
22     bienvenida = s.recv(1024).decode("utf-8")
23     print(bienvenida, end="")
24
25     while True:
26         linea = input("> ").strip()
27         if not linea:
28             continue
29         s.sendall((linea + "\n").encode("utf-8"))
30         respuesta = s.recv(1024).decode("utf-8")
31         print(respuesta, end="")
32         if linea.upper() == "SALIR":
33             break
34
35
36 if __name__ == "__main__":
37     main()

```

El cliente admite host y puerto como argumentos de línea de comandos, con valores por defecto para ejecución local (127.0.0.1:5050), de forma que el mismo script sirve tanto para probar el servidor en local como para conectarse al servidor ya desplegado, indicando el host y puerto públicos del proxy TCP.

5.4. Prueba automática multicliente (prueba_multicliente.py)

Listing 3: Prueba automática de dos clientes concurrentes

```

1  """
2  Prueba automatica del servidor: lanza dos clientes en paralelo (dos hilos)
3  que hablan con el servidor al mismo tiempo, para comprobar que atiende a
4  varias conexiones simultaneas y que cada una mantiene su propia
5  conversacion sin mezclarse con la otra.
6
7  Requiere que server.py este en ejecucion en 127.0.0.1:5050 antes de
8  lanzar este script.
9
10 Uso:
11     Terminal 1> python server.py
12     Terminal 2> python prueba_multicliente.py
13 """
14
15 import socket
16 import threading
17 import time
18
19 HOST, PORT = "127.0.0.1", 5050
20
21
22 def run(nombre, comandos):
23     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
24         s.connect((HOST, PORT))
25         print(f"[{nombre}] << {s.recv(1024).decode().strip()}")
26         for c in comandos:
27             s.sendall((c + "\n").encode())
28             resp = s.recv(1024).decode().strip()
29             print(f"[{nombre}] > {c}")

```

```

30     print(f"[{nombre}] << {resp}")
31     time.sleep(0.1)
32
33
34 if __name__ == "__main__":
35     cliente1 = threading.Thread(target=run, args=("cliente1", [
36         "HOLA",
37         "HORA",
38         "ECO hola mundo",
39         "CORREO profesor@example.com|Entrega practica|Adjunto el servidor de sockets",
40         "COMANDO_INVALIDO",
41         "CORREO formato_incorrecto_sin_separadores",
42         "SALIR",
43     ]))
44     cliente2 = threading.Thread(target=run, args=("cliente2", [
45         "HOLA",
46         "ECO soy el segundo cliente, en paralelo",
47         "SALIR",
48     ]))
49     cliente1.start()
50     cliente2.start()
51     cliente1.join()
52     cliente2.join()

```

Este script lanza dos clientes en hilos separados, cada uno con su propia secuencia de comandos, para comprobar que el servidor mantiene correctamente conversaciones independientes con clientes simultáneos sin mezclar sus respuestas. Está preparado para ejecutarse contra una instancia local del servidor (127.0.0.1:5050).

6. Desarrollo de la práctica: despliegue

6.1. Elección de plataforma

Antes de desplegar, se compararon varias plataformas de hosting gratuitas atendiendo a dos requisitos imprescindibles para esta práctica: exponer un puerto TCP en bruto (no solo HTTP) y permitir tráfico saliente hacia servicios externos.

Plataforma	Tarjeta requerida	TCP en bruto	Observación
Railway (Full Trial vía GitHub)	No	Sí (TCP Proxy)	Elegida para esta práctica
Fly.io	Sí (trial sin cargo)	Sí, nativo	Alternativa preparada, no usada
Render.com (plan gratuito)	No	Solo HTTP(S)	Descartada (no sirve para sockets)
PythonAnywhere (plan gratuito)	No	No	Descartada (no permite servidor p...

Cuadro 2: Comparativa de plataformas de despliegue consideradas

Se optó por Railway por no requerir tarjeta de crédito y por soportar un proxy TCP nativo hacia el puerto interno del contenedor.

6.2. Método de despliegue

El despliegue se realizó con el CLI oficial de Railway (`railway up`), que empaqueta el directorio del proyecto y lo sube directamente a la plataforma, sin pasar por un repositorio

Git intermedio. El registro real de la primera construcción exitosa de la imagen muestra que Railway detectó y utilizó el `Dockerfile` del proyecto para construir la imagen del contenedor:

```
Deployment: https://railway.com/project/.../service/...
scheduling build on Metal builder "builder-mpbfot"
[internal] load build definition from Dockerfile
[internal] load metadata for docker.io/library/python:3.11-slim
[1/3] FROM docker.io/library/python:3.11-slim@sha256:e41613d4...
[2/3] WORKDIR /app
[3/3] COPY server.py .
exporting to docker image format
image push
Deploy complete
Servidor escuchando en 0.0.0.0:5050 (MODULO_PRUEBA=inactivo)
Starting Container
```

Este es un resultado real observado durante el despliegue, no una salida teórica: confirma que la imagen se construyó a partir de la versión de tres pasos del `Dockerfile` (imagen base `python:3.11-slim`, directorio de trabajo `/app`, copia de `server.py`) y que el proceso arrancó correctamente escuchando en el puerto interno 5050.

6.3. Configuración del proxy TCP

En el panel de Railway, dentro de *Settings > Networking > TCP Proxy*, se configuró un proxy que expone el servicio bajo el host público `mainline.proxy.rlwy.net`, puerto 59568, redirigiendo hacia el puerto interno 5050 del contenedor. Este es el destino que usa el cliente para conectarse al servidor ya desplegado:

```
python client.py mainline.proxy.rlwy.net 59568
```

7. Resultados

Esta sección recoge los resultados reales obtenidos a lo largo de las distintas iteraciones del despliegue, en el orden cronológico en que se produjeron. Se distingue explícitamente lo que se esperaba teóricamente de lo que se obtuvo realmente en cada prueba, tal como exige el análisis de una práctica experimental.

#	Comando / acción	Resultado esperado (teórico)	Resultado observado (real)
1	Conectar el cliente antes de crear el TCP Proxy	Conexión aceptada o rechazada explícitamente	<code>TimeoutError</code> en <code>connect()</code> : no había ninguna ruta pública configurada
2	Conectar tras crear el TCP Proxy (<code>mainline.proxy.rlwy.net:59568</code>)	Conexión aceptada, mensaje de bienvenida	<code>conexion OK</code> seguido del mensaje BIENVENIDO...
3	HOLA	HOLA, cliente	HOLA, cliente
4	HORA	Fecha y hora actuales del servidor	2026-09-28 09:45:55

#	Comando / acción	Resultado esperado (teórico)	Resultado observado (real)
5	ECO esto es una prueba	esto es una prueba	esto es una prueba
6	CORREO vía SMTP puerto 587 (sin corrección de IPv4)	Conexión SMTP y envío del correo	ERROR error de red: [Errno 101] Network is unreachable
7	CORREO vía SMTP puerto 587 (con resolución forzada a IPv4)	Envío correcto, o fallo por filtrado del puerto	ERROR error de red: timed out
8	CORREO vía SMTP puerto 465 (TLS implícito)	Envío correcto si el puerto 465 no estuviera filtrado	ERROR error de red: timed out
9	CORREO vía API de Resend, antes de propagar la variable RESEND_API_KEY	Simulación en MODO_PRUEBA	OK simulado (MODO_PRUEBA activo, sin RESEND_API_KEY)
10	CORREO vía API de Resend, con clave activa, sin User-Agent propio	Correo enviado correctamente	ERROR error HTTP 403 de Resend: error code: 1010
11	CORREO vía API de Resend, con User-Agent propio	Correo enviado correctamente	OK correo enviado (id Resend: 01a0e779-5425-731d-933e-2ca708b17348)

Cuadro 3: Resultados reales observados en las sucesivas pruebas del comando **CORREO**

El último resultado (fila 11) constituye la confirmación de éxito a nivel de protocolo: la API de Resend respondió con un código HTTP 200 y un identificador de mensaje válido, lo que indica que aceptó la petición de envío. La verificación visual de la recepción del correo en la bandeja de entrada del destinatario queda pendiente de confirmación manual por parte del estudiante, y no se documenta aquí por no disponer de esa evidencia directa en el momento de redactar esta memoria.

8. Análisis

8.1. Sobre la naturaleza de los errores de red observados

Un aspecto especialmente instructivo de esta práctica es que los distintos fallos de red obtenidos en las filas 1, 6, 7-8 y 10 de la tabla 3 no son intercambiables entre sí: cada tipo de error indica un problema distinto y a un nivel distinto de la comunicación, y confundirlos habría llevado a diagnósticos equivocados.

- **TimeoutError en connect() (fila 1):** no existía ninguna ruta pública hacia el servidor, porque el proxy TCP no estaba creado. El sistema operativo del cliente

enviaba el intento de conexión, pero nunca llegaba respuesta porque no había ningún proceso escuchando en ese host:puerto público.

- **[Errno 101] Network is unreachable (fila 6)**: este es un error de *enrutamiento*, devuelto de forma inmediata por el sistema operativo, no tras esperar una respuesta. Indica que el contenedor no tenía ninguna ruta de red hacia la dirección de destino. La causa identificada fue que `smtp.gmail.com` publica tanto una dirección IPv4 como una IPv6, y el contenedor de Railway anunciaba soporte IPv6 en su pila de red sin tener una ruta IPv6 real hacia el exterior: al intentar conectar por IPv6, el fallo era inmediato.
- **timed out (filas 7 y 8)**: a diferencia del error anterior, un timeout significa que el paquete sí se envió (ya no había problema de enrutamiento, tras forzar IPv4), pero nadie respondió en el plazo configurado. En un proveedor de hosting, esto es característico de un cortafuegos de salida que descarta silenciosamente el tráfico hacia determinados puertos, sin devolver ningún rechazo explícito.
- **HTTP 403 (error code: 1010) (fila 10)**: este ya no es un fallo de red en absoluto, sino un rechazo a nivel de aplicación: la conexión TCP y la negociación TLS se completaron con éxito, y el servidor de Cloudflare que protege la API de Resend llegó a procesar la petición HTTP, pero la rechazó por considerar sospechosa la cabecera `User-Agent` por defecto de la biblioteca cliente utilizada.

Esta progresión (fallo de enrutamiento → fallo de conexión por filtrado → fallo de autorización a nivel de aplicación) ilustra cómo, en sistemas distribuidos reales, un mismo síntoma superficial ("no funciona") puede tener causas en capas completamente distintas, y que la estrategia correcta de diagnóstico consiste en aislar en qué capa se produce exactamente el fallo antes de intentar corregirlo.

8.2. Sobre el filtrado de puertos SMTP en plataformas de hosting

El bloqueo del tráfico saliente hacia los puertos 587 y 465 no está documentado explícitamente en la plataforma utilizada, pero es una práctica extendida entre proveedores de hosting gratuito o de bajo coste, con el objetivo de impedir que sus contenedores se utilicen para el envío masivo de correo no deseado (spam). Esto tiene una implicación práctica relevante para el diseño de sistemas distribuidos desplegados en la nube: no puede asumirse que un servicio tenga conectividad saliente irrestricta simplemente porque tenga conectividad entrante; cada dirección del tráfico (entrante/saliente) y cada puerto pueden estar sujetos a políticas de filtrado independientes, y conviene comprobarlo empíricamente en el entorno de despliegue real, en lugar de asumirlo a partir del comportamiento en el entorno de desarrollo local.

8.3. Sobre delegar en un servicio externo vía HTTP en lugar de SMTP directo

Sustituir la conexión SMTP directa por una petición a una API HTTP no altera el patrón de diseño de la práctica (seguir delegando en un sistema de mensajería externo), pero sí cambia el protocolo de transporte hacia ese sistema. Desde el punto de vista de la asignatura, es un buen ejemplo de cómo la elección de protocolo de un sistema distribuido

está condicionada no solo por requisitos funcionales, sino también por restricciones del entorno de despliegue: HTTP(S) sobre el puerto 443 es, en la práctica, el protocolo con menos probabilidad de encontrarse filtrado en cualquier red, precisamente porque es el que sostiene la navegación web general, mientras que protocolos más especializados como SMTP en puertos no estándar son más susceptibles de estar restringidos.

8.4. Sobre la concurrencia del servidor

El diseño con un hilo por conexión permite que el servidor atienda a varios clientes simultáneamente sin que uno bloquee al resto, lo cual queda demostrado a nivel de diseño por la estructura de `prueba_multicliente.py` (dos clientes conversando en paralelo, cada uno con su propia secuencia de comandos y sin interferir con el otro). Al no disponerse todavía de una ejecución documentada de ese script contra el servidor, esta conclusión se sostiene en el análisis del código (cada conexión aceptada se delega de inmediato a un hilo independiente, sin que el bucle de `accept()` quede bloqueado) y queda pendiente de confirmación experimental explícita.

9. Problemas encontrados

Problema 1: proxy TCP no configurado

Al intentar conectar el cliente contra el host y puerto públicos, la conexión fallaba con `TimeoutError`. No existía ningún TCP Proxy creado en el panel de Railway que expusiera el puerto interno 5050 del contenedor.

Problema 2: proxy TCP mal configurado

Tras crear un primer proxy, se detectaron dos proxies simultáneos: uno correcto (`mainline.proxy.rlwy.` → `:5050`) y otro apuntando a un puerto interno (`:59568`) en el que no escuchaba ningún proceso, por lo que ese segundo proxy no conducía a ningún sitio.

Problema 3: Network is unreachable al conectar con el servidor SMTP

Al ejecutar el comando `CORREO`, la conexión SMTP fallaba de inmediato con `[Errno 101] Network is unreachable`, un error de enrutamiento a nivel de sistema operativo, no un simple fallo de aplicación.

Problema 4: timed out tras corregir el enrutamiento IPv4

Una vez resuelto el problema anterior, la conexión SMTP (tanto en el puerto 587 como en el 465) agotaba el tiempo de espera sin respuesta, indicando un filtrado del tráfico saliente por parte de la plataforma de hosting hacia esos puertos concretos.

Problema 5: falta de repositorio Git para actualizar el despliegue

El servicio en Railway se había desplegado originalmente con el CLI (`railway up`) y no a través de un repositorio de GitHub, por lo que no existía ningún repositorio Git local desde el que aplicar y subir las correcciones de código.

Problema 6: rechazo HTTP 403 al llamar a la API de Resend

Tras sustituir SMTP por la API HTTP de Resend, la primera petición real (ya con la clave de API configurada) fue rechazada con un código HTTP 403 y el mensaje `error code: 1010`, propio de la protección de Cloudflare situada delante de esa API.

10. Soluciones adoptadas

Solución 1

Se creó un TCP Proxy en *Settings > Networking* del servicio en Railway, apuntando explícitamente al puerto interno 5050, que es el que usa `server.py` por defecto.

Solución 2

Se identificó cuál de los dos proxies era el correcto comparando su puerto interno de destino con el puerto real de escucha del servidor, y se recomendó eliminar el proxy sobrante para evitar compartir por error un destino no funcional.

Solución 3

Se interceptó la función `socket.getaddrinfo` para forzar que solo devolviera direcciones IPv4, evitando que la biblioteca de sockets intentara primero la dirección IPv6 de `smtp.gmail.com`, hacia la que el contenedor no tenía ruta real. Esta solución quedó posteriormente obsoleta al abandonar SMTP en favor de la API HTTP (solución 6), y se retiró del código.

Solución 4

Al confirmarse que el filtrado afectaba a ambos puertos SMTP habituales (587 y 465), se descartó la vía SMTP directa como no viable en esta plataforma, y se adoptó la solución 6 (API HTTP externa) como alternativa arquitectónica, en lugar de seguir intentando distintos puertos o parámetros SMTP.

Solución 5

Se confirmó, mediante la variable de entorno de Railway y `railway whoami`, que el servicio estaba vinculado al proyecto a través de sesión de CLI. Las actualizaciones de código posteriores se aplicaron directamente sobre los archivos locales del proyecto y se subieron con `railway up`, sin necesidad de un repositorio Git intermedio.

Solución 6

Se sustituyó el envío por SMTP directo por una petición HTTPS POST a la API de Resend (`https://api.resend.com/emails`), autenticada con una clave de API pasada como variable de entorno (`RESEND_API_KEY`). Al usar el puerto 443, esta vía no está sujeta al filtrado detectado en los puertos SMTP.

Solución 7

Se añadió una cabecera **User-Agent** propia y descriptiva a la petición HTTP hacia la API de Resend, evitando la firma por defecto de `urllib` (`Python-urllib/x.y`) que la protección de Cloudflare identificaba como tráfico automatizado sospechoso.

11. Conclusiones

Esta práctica ha permitido trabajar de forma directa con los fundamentos de la comunicación mediante sockets TCP: el ciclo de vida de un socket servidor, la necesidad de delimitar mensajes sobre un transporte orientado a flujo de bytes, y la concurrencia mediante hilos para atender a varios clientes sin bloquear el servicio. Sobre esa base, el comando **CORREO** ha servido para ilustrar un patrón habitual en sistemas distribuidos: delegar en un sistema de mensajería externo (un MOM) para completar una funcionalidad, en lugar de implementarla íntegramente en el propio servicio.

Más allá del diseño inicial, el proceso real de despliegue en una plataforma de hosting en la nube ha resultado especialmente formativo, precisamente por las dificultades encontradas. Se ha comprobado empíricamente que un entorno de despliegue introduce restricciones de red (filtrado de puertos salientes, problemas de enrutamiento IPv6, protecciones a nivel de aplicación como Cloudflare) que no existen en el entorno de desarrollo local y que no siempre están documentadas de antemano. Diagnosticar cada uno de estos problemas ha requerido distinguir con precisión el tipo de error obtenido en cada caso (fallo de enrutamiento, timeout por filtrado, rechazo HTTP a nivel de aplicación), en línea con la idea, señalada en el propio temario de la asignatura, de que en sistemas asíncronos y distribuidos no siempre es trivial distinguir entre causas distintas de un mismo síntoma aparente, y de que conviene basar el diagnóstico en la evidencia concreta que ofrece cada tipo de error, y no en suposiciones.

Como línea de continuación, quedaría pendiente ejecutar y documentar formalmente la prueba automática multicliente (`prueba_multicliente.py`) contra el servidor ya desplegado, así como confirmar de forma visual la recepción del correo electrónico enviado en la última prueba realizada.

12. Bibliografía

- Python Software Foundation. *Documentación del módulo `socket`*. <https://docs.python.org/3/library/socket.html>
- Python Software Foundation. *Documentación del módulo `urllib.request`*. <https://docs.python.org/3/library/urllib.request.html>
- Python Software Foundation. *Documentación del módulo `threading`*. <https://docs.python.org/3/library/threading.html>
- Postel, J. *RFC 793: Transmission Control Protocol*. IETF, 1981. <https://www.rfc-editor.org/rfc/rfc793>
- Klensin, J. *RFC 5321: Simple Mail Transfer Protocol*. IETF, 2008. <https://www.rfc-editor.org/rfc/rfc5321>
- Hoffman, P. *RFC 3207: SMTP Service Extension for Secure SMTP over Transport Layer Security*. IETF, 2002. <https://www.rfc-editor.org/rfc/rfc3207>

- Railway. *Documentación de despliegue y redes (TCP Proxy)*. <https://docs.railway.com>
- Resend. *Documentación de la API de envío de correo*. <https://resend.com/docs>
- Apuntes de la asignatura: *T1 - Fundamentos de los sistemas distribuidos, T2 - Comunicación, T3 - Coordinación* (material proporcionado en la asignatura).

13. Anexos

13.1. Dockerfile

Listing 4: Dockerfile usado para construir la imagen del contenedor

```
FROM python:3.11-slim
WORKDIR /app
COPY server.py .
EXPOSE 5050
CMD ["python", "server.py"]
```

13.2. Procfile

Listing 5: Procfile (comando de arranque alternativo, vía Nixpacks)

```
web: python server.py
```

13.3. requirements.txt

Listing 6: requirements.txt: sin dependencias externas

```
# El servidor y el cliente solo usan la biblioteca estandar de Python
# (socket, threading, json, os, urllib). No hay dependencias externas.
# Este archivo existe para que las plataformas de despliegue detecten
# el proyecto como aplicacion Python sin ambigüedad.
```

13.4. runtime.txt

Listing 7: runtime.txt: version de Python

```
python-3.11
```

13.5. Registro real del despliegue exitoso (railway up)

Listing 8: Salida real del último despliegue, tras aplicar todas las correcciones

```
railway up
Indexed
Compressed [=====] 100%
Uploaded
```

```
Deployment: https://railway.com/project/83361212-2a9d-473f-ab43-349007114f18/service/2
b0a7868-eb69-430e-bf02-bf07d696cf55?id=7776a4e3-d74c-495b-8a26-e6ac5405a291
scheduling build on Metal builder "builder-mpbfot"
fetched snapshot sha256:2567
bb5ed54b5401e1faa4e2dae127c80d5b270d08931d8067972cb9c76b365e (7.7 kB bytes)
fetching snapshot
unpacking archive
[internal] load build definition from Dockerfile
[internal] load metadata for docker.io/library/python:3.11-slim
[internal] load .dockerignore
[3/3] COPY server.py .
[internal] load build context
[2/3] WORKDIR /app
[1/3] FROM docker.io/library/python:3.11-slim@sha256:
e41613d42d4891e4930f79523f93f81bbc7632584ec65e36ab055f41a800b41e
exporting to docker image format
image push
Deploy complete
Starting Container
Servidor escuchando en 0.0.0.0:5050 (MOD0_PRUEBA=inactive)
```

13.6. Sesión real del cliente confirmando el envío de correo

Listing 9: Sesión real del cliente contra el servidor desplegado, con envío de correo confirmado

```
py client.py mainline.proxy.rlwy.net 59568
Conectando a mainline.proxy.rlwy.net:59568...
BIENVENIDO. Comandos: HOLA | HORA | ECO <msg> | CORREO <destino>|<asunto>|<cuerpo> |
SALIR
> CORREO guillerromo@gmail.com|Prueba via Resend|Correo enviado a traves de la API
HTTP de Resend
OK correo enviado (id Resend: 01a0e779-5425-731d-933e-2ca708b17348)
```